AASD 4016
Full Stack Data Science Systems

Applied AI Solutions Developer Program

Module 3
Clients

Vejey Gandyer

Agenda

- Clients
- Curl
- POSTMAN
- Python Web Client
- Templates
- Receive Arguments
- Examples

Clients

Curl

POSTMAN

Python

Flask

Installing Flask

```
pip install Flask
```

Hello Flask Application (hello.py)

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

Flask

Running a Flask Application

```
export FLASK_APP=hello
flask run
```

Set Flask Environment

```
export FLASK_ENV=development
```

Project Structure

```
/home/user/Projects/flask-tutorial
├── flaskr/
│   ├── __init__.py
│   ├── db.py
│   ├── schema.sql
│   ├── auth.py
│   ├── blog.py
│   ├── templates/
│   │   ├── base.html
│   │   ├── auth/
│   │   │   ├── login.html
│   │   │   └── register.html
│   │   └── blog/
│   │       ├── create.html
│   │       ├── index.html
│   │       └── update.html
│   └── static/
│       └── style.css
├── tests/
│   ├── conftest.py
│   ├── data.sql
│   ├── test_factory.py
│   ├── test_db.py
│   ├── test_auth.py
│   └── test_blog.py
├── venv/
├── setup.py
└── MANIFEST.in
```

Current Flask project structure

Project Structure

```
app.py
helper(s).py [Optional]
```

Iris Classifier

Notebook to Script

Script to Flask

Load model and keep it up for inference

Infer from the model

Test the endpoint using Curl, POSTMAN

Build a python web client

Test the client

Expected Flask project structure

Project Structure

```
app.py
helper(s).py [Optional]
templates/
    index.html
    result.html
static/
    Style.css [Optional]
```

Curl

```
curl http://localhost:5000/
```

```
curl http://localhost:5000/classify
```

Future Curl Calls (to be developed)

```
curl http://localhost:5000/
```

```
curl http://localhost:5000/classify?sl=5.1&sw=3.2&pl=3.5&pw=4.2
```

POSTMAN

www.postman.com

Download and install the application

Follow along the demo

POSTMAN

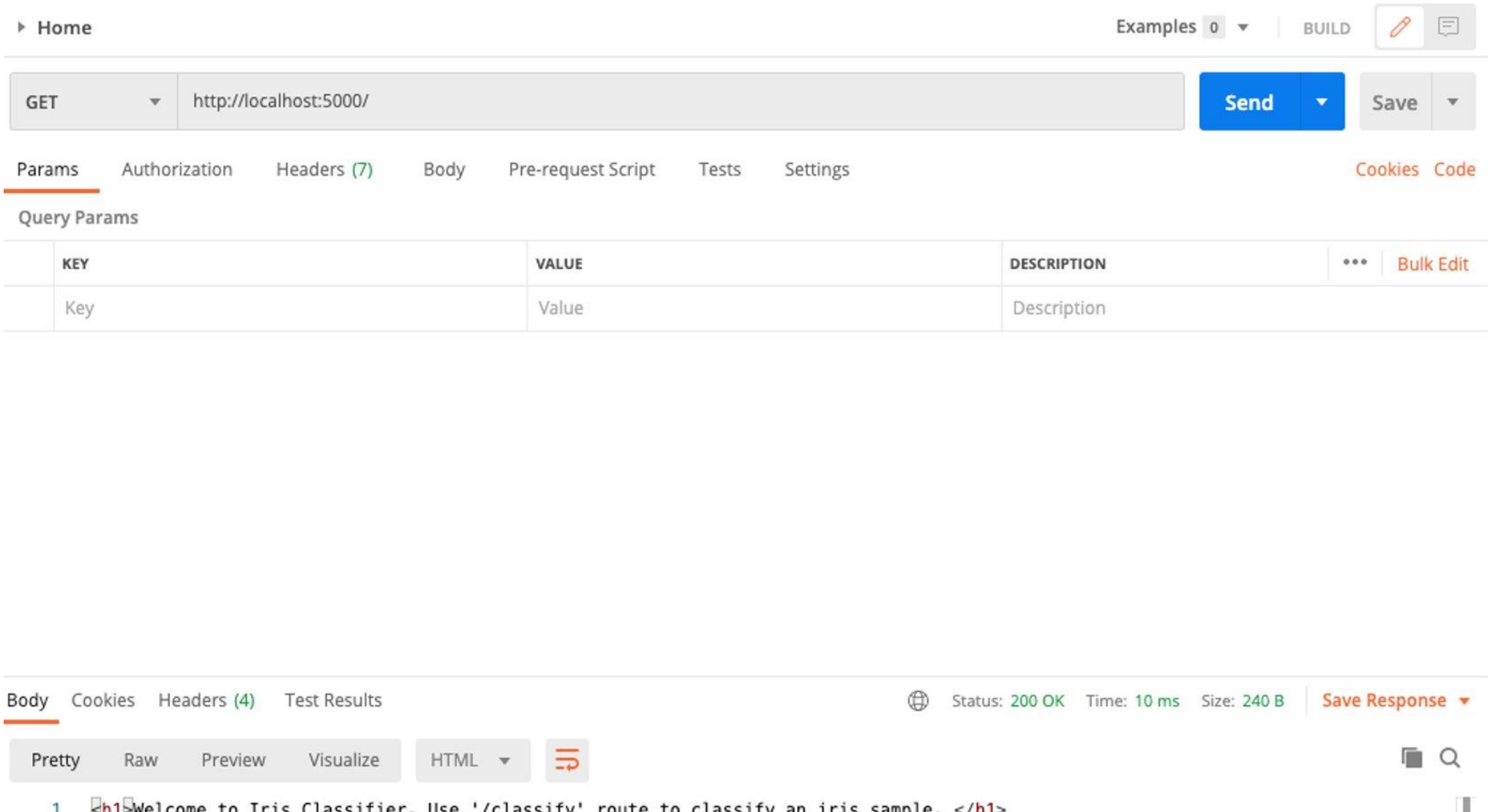
Create a Collection

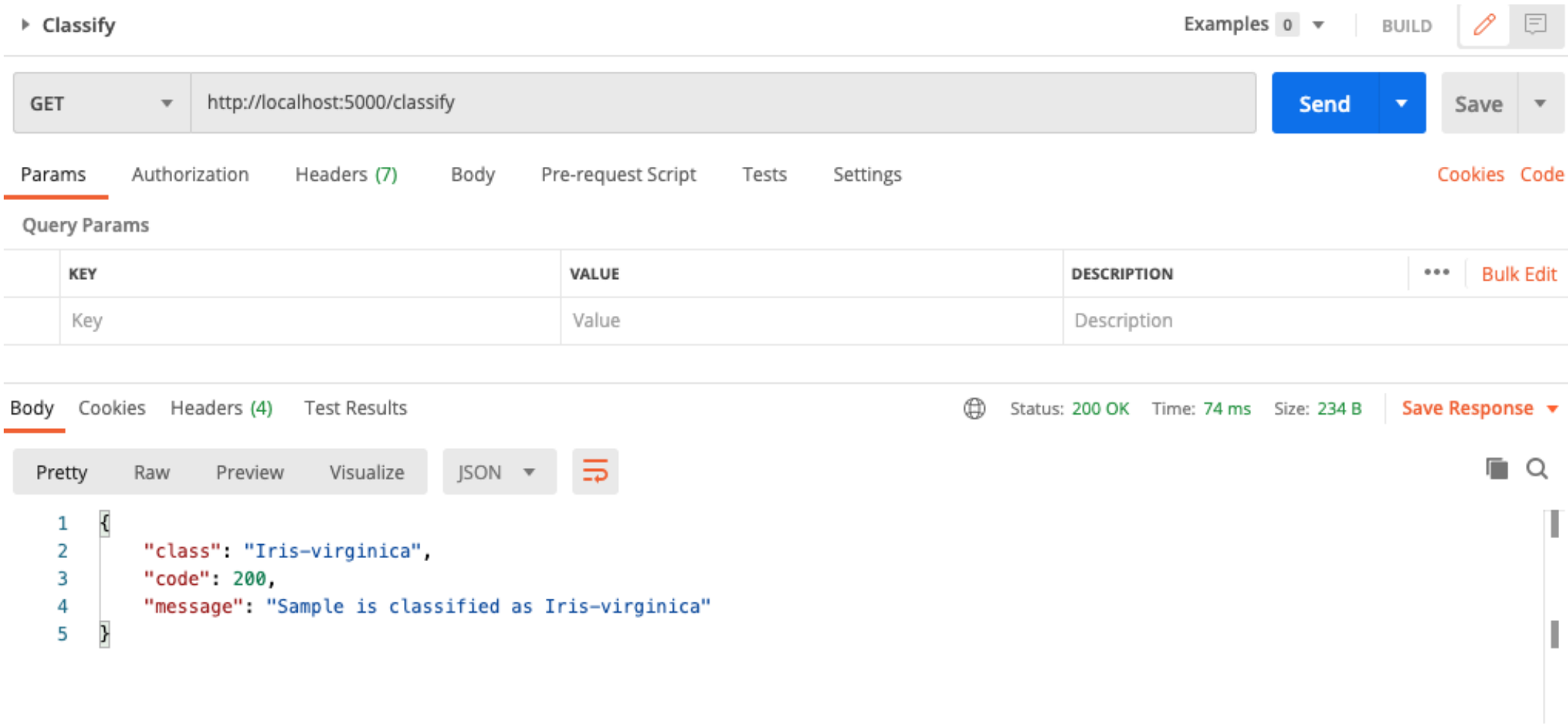
Create endpoint (either GET / POST)

Give the URL for the endpoint

Give the parameters for the endpoint (if any)

Get the response


Home
Examples 0
BUILD
GET
http://localhost:5000/
Send
Save
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Code
Query Params
KEY
VALUE
DESCRIPTION
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (4)
Test Results
Status: 200 OK
Time: 10 ms
Size: 240 B
Save Response
Pretty
Raw
Preview
Visualize
HTML
1 <h1>Welcome to Iris Classifier. Use '/classify' route to classify an iris sample. </h1>


Classify
Examples 0
BUILD
GET
http://localhost:5000/classify
Send
Save
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Code
Query Params
KEY
VALUE
DESCRIPTION
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (4)
Test Results
Status: 200 OK
Time: 74 ms
Size: 234 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"class": "Iris-virginica",
"code": 200,
"message": "Sample is classified as Iris-virginica"
}

Python Web Client

Create Web client with html

Include the input fields within html

Stylize the html elements with css (Optional)

Connect the button with corresponding Flask function

HTTP Request Methods

GET

POST

PUT

HEAD

DELETE

GET

- GET is used to request data from a resource
- Query String is sent in the URL of a GET request
  - Developer Tools
  - /test_endpoint?name=subash&age=41

POST

- POST is used to send data to a server to create / update a resource
- Data sent to the server is stored in the request body of the HTTP request
  - Developer Tools
  - POST /test_endpoint HTTP/1.1
    Host: iamsubash.com
    name=subash&age=41

PUT

- PUT is used to send data to a server to create / update a resource
- Calling PUT request multiple times produce the same result
- Calling POST request multiple times produce multiple resources

HEAD

- HEAD is identical to GET, but without the response body
- HEAD requests are useful for checking what a GET request will return before actually making an actual GET request
  - Before downloading a very large file

DELETE

- DELETE deletes the specified resource

GET vs POST

	GET	POST
BACK button/Reload	Harmless	Data will be re-submitted (the browser should alert the user that the data are about to be re-submitted)
Bookmarked	Can be bookmarked	Cannot be bookmarked
Cached	Can be cached	Not cached
Encoding type	application/x-www-form-urlencoded	application/x-www-form-urlencoded or multipart/form-data. Use multipart encoding for binary data
History	Parameters remain in browser history	Parameters are not saved in browser history
Restrictions on data length	Yes, when sending data, the GET method adds the data to the URL; and the length of a URL is limited (maximum URL length is 2048 characters)	No restrictions
Restrictions on data type	Only ASCII characters allowed	No restrictions. Binary data is also allowed
Security	GET is less secure compared to POST because data sent is part of the URL Never use GET when sending passwords or other sensitive information!	POST is a little safer than GET because the parameters are not stored in browser history or in web server logs
Visibility	Data is visible to everyone in the URL	Data is not displayed in the URL

Flask HTTP Methods

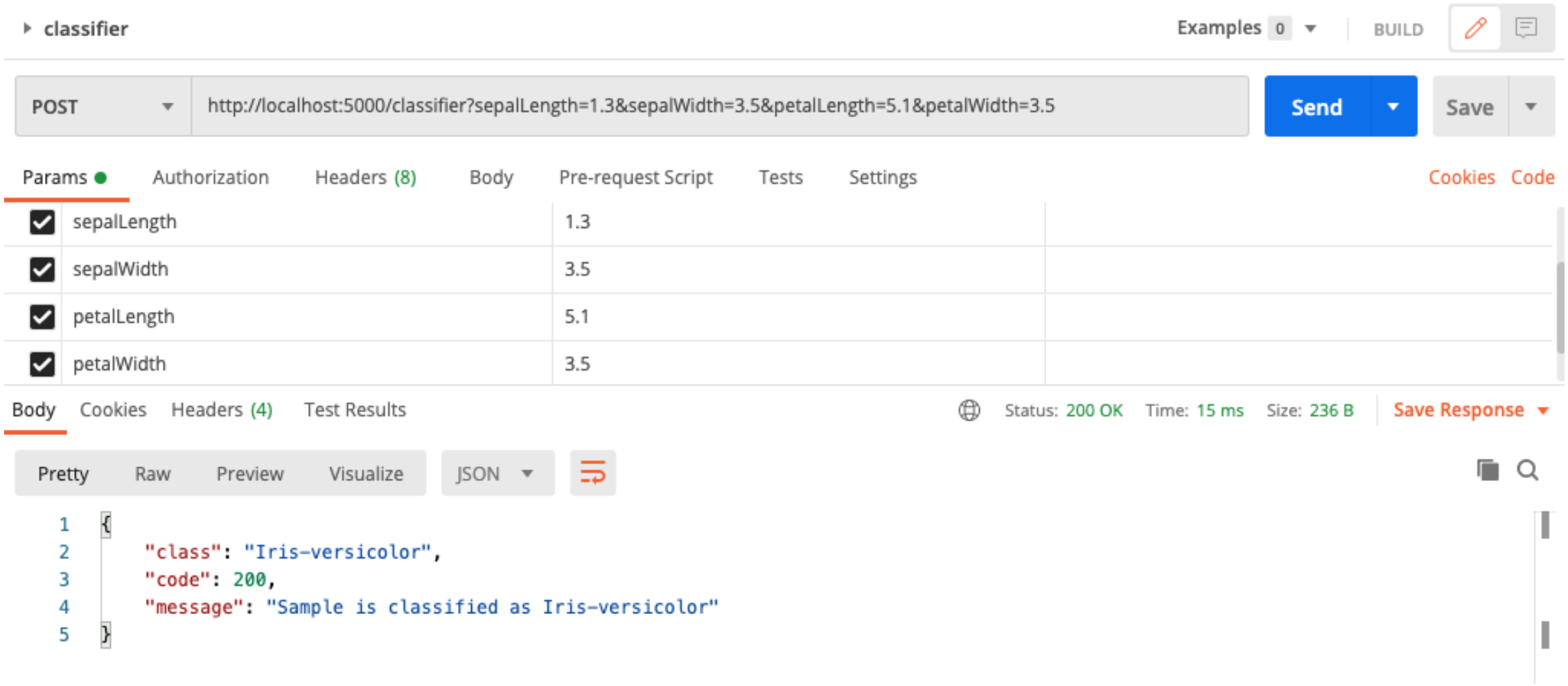
HTTP Methods

```
from flask import request


@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()
```

Receive Arguments

```
from flask import request


@app.route('/classifier', methods=['GET', 'POST'])
def classifier():
    sepalLength = float(request.args['sepalLength'])
    sepalWidth = float(request.args['sepalWidth'])
    petalLength = float(request.args['petalLength'])
    petalWidth = float(request.args['petalWidth'])
```

Receive Arguments



Index.html

```
<html>
   <body>
      <h1>IRIS Classifier</h1>
   </body>
</html>
```

Result.html

```
<html>
  <body>
    <h1>IRIS Classifier</h1>
    {{ result }}
  </body>
</html>
```

Rendering Templates

```
from flask import render_template

@app.route('/hello)
def hello(name=None):
    return render_template('index.html', name=name)

@app.route('/classifier, methods=['GET', 'POST'])
def classifier():
     …
     …

      return render_template('result.html', result=result)
```

Receive Arguments: Checkbox

```
<form action="/classifier" method="post" enctype="multipart/form-data">
  <h4> Choose a choice </h4>
  <label>Yes</label><input type="checkbox" name="yes" default=False />
  <label>No</label><input type="checkbox" name="no" default=False />
```

```
@app.route('/classifier, methods=['GET', 'POST'])
def classifier():
    if request.method == 'POST':
        yes_choice = request.form.get('yes')
        ...
```

Receive Arguments: Radio Button

```
<form action="/classifier" method="post" enctype="multipart/form-data">
<h4> Choose a Gender </h4> <span style="color:red;"> *</span>
<label>Gender</label> <span style="color:red;"> *</span>
<label>MALE</label><input type="radio" name="gender" id="male"
  value="MALE">
<label>FEMALE</label><input type="radio" name="gender" id="female"
  value="FEMALE">
```

```
@app.route('/classifier, methods=['GET', 'POST'])
def classifier():
    if request.method == 'POST':
        gender = request.form.get('gender')
        ...
```

Receive Arguments: File uploads

```
<form action="/classifier" method="post" enctype="multipart/form-
  data">
<h4> Choose a CSV file </h4> <span style="color:red;"> *</span>
<input type="file" name="csv" />


@app.route('/classifier, methods=['GET', 'POST'])
def classifier():
    if request.method == 'POST':
        f = request.files['csv']
        f.save('/var/www/uploads/uploaded_file.txt')
    ...
```

Receive Arguments: Secure File uploads

```
from werkzeug.utils import secure_filename


@app.route('/classifier, methods=['GET', 'POST'])
def classifier():
    if request.method == 'POST':
        f = request.files['the_file']
                                f.save(f'/var/www/uploads/
{secure_filename(f.filename)}')
    ...
```

style.css [Optional]

All together for a Python Web Client

```
index.html

app.py

style.css [ Optional ]
```

Testing Python Web Client

Test all endpoints

- Curl
- Postman

Further Reading

Flask Tutorial

https://flask.palletsprojects.com/en/2.0.x/tutorial/